Azure Monitor & Log Analytics — Foundation, Alerts & Dashboards

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards 4

The problem . . . 4

The solution . . . 4

What's included . . . 4

Business value . . . 5

Effort and boundary . . . 5

1. Solution Overview — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards 5

What this solution is . . . 5

How the technology works . . . 5

What this solution builds (the components) . . . 6

The standard deployment model . . . 7

Scope . . . 7

What is out of scope . . . 7

Authentication and connections used . . . 7

Limitations (be honest with buyers) . . . 7

2. Requirements Guide — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards 8

2.1 Monitoring-capable rights at the target scope . . . 8

2.2 A Log Analytics workspace design decision (retention, region, tier) . . . 8

2.3 Azure Monitor Agent (AMA) + Data Collection Rule readiness (for VM/container insights) . . . 8

2.4 Azure Policy managed-identity permissions for DeployIfNotExists . . . 9

2.5 Notification channel details (action group) . . . 9

2.6 A defined pilot resource group . . . 9

2.7 Az PowerShell + Bicep tooling on the operator workstation . . . 9

Checklist . . . 10

Global Requirements — Observability (category-level) 10

G1. Target cloud/estate with sufficient rights . . . 10

G2. A telemetry destination (workspace / monitoring account) . . . 10

G3. Agent / collection rollout channel . . . 10

G4. IaC tooling and source control . . . 11

G5. Secrets management . . . 11

G6. Ingestion cost plan . . . 11

G7. Alerting and notification targets . . . 11

G8. A pilot scope . . . 11

3. Implementation Guide — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards 11

Step 1 — Assess: baseline the estate and existing telemetry . . . 12

Step 2 — Configure: deploy the workspace foundation (Bicep) . . . 12
Step 3 — Configure: stage the diagnostics policy (audit / DoNotEnforce first) . . . 13
Step 4 — Pilot: onboard the pilot resource group . . . 13
Step 5 — Pilot: wire alerts + action group . . . 14
Step 6 — Validate the pilot (the gates) . . . 14
Step 7 — Build the workbooks and dashboards . . . 14
Step 8 — Expand: remediate diagnostics + roll out alerts estate-wide . . . 14
5. Testing & Validation — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards 15
Test matrix . . . 15
Automated checks . . . 17
Evidence to capture . . . 17
Exit criteria (before Expand / estate-wide remediation) . . . 17
6. Troubleshooting — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards 17
No telemetry in the workspace / tables empty . . . 17
Policy assigned but nothing is being deployed/remediated . . . 18
Metric alert never fires . . . 18
Log (scheduled query) alert never fires . . . 18
Notifications don't arrive . . . 18
Ingestion cost spikes unexpectedly . . . 19
VM insights / container insights not populating . . . 19
Workbook or dashboard shows errors / no data . . . 19
"Access denied" running the scripts or assigning policy . . . 19
7. Best Practices — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards 20
Collect what you will act on . . . 20
One workspace, governed . . . 20
Diagnostics by policy, not by hand . . . 20
Alerts that page, not spam . . . 20
Secrets and access hygiene . . . 21
Version & cost discipline . . . 21
Where to go next (adjacent catalog areas) . . . 21
8. Deliverables Summary — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards 21
What you receive . . . 22
Outcomes . . . 23
Assumptions & boundaries . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards

The problem

Azure resources emit rich telemetry — platform metrics, resource logs, guest OS performance, activity records — but most estates never collect or act on it. Diagnostic settings are enabled on a handful of resources by hand (or not at all), there is no central workspace, no alert rules that page the right team, and no dashboards to look at. The result is predictable: problems surface as **outages and user complaints** instead of as **alerts with context**. When something does break, no one can answer "when did this start, what changed, and what else is affected?" because the logs were never gathered.

The solution

This solution builds an **Azure Monitor observability foundation** — an ops/platform monitoring layer, distinct from a security SIEM — in five parts:

1. **A designed Log Analytics workspace** — one governed telemetry destination with deliberate

retention, **access (RBAC)**, and a **cost/tier** posture, rather than logs scattered across ad-hoc workspaces.

1. **Diagnostic settings at scale via Azure Policy** — a policy/initiative assignment that

automatically routes resource logs and platform metrics from every in-scope resource to the workspace, so new resources are covered without anyone remembering to click.

1. **Metric and log alert rules with action groups** — alerts that fire on real conditions (a VM stops

heart-beating, CPU/latency breaches threshold, a resource is unhealthy) and **route to the right responder** (email, Teams/Slack, PagerDuty) with context.

1. **VM insights and container insights** — guest-level performance and Kubernetes/container telemetry

via the **Azure Monitor Agent** and **Data Collection Rules**, on the resources that need it.

1. **Workbooks and dashboards** — an **ops overview** workbook and an **executive** dashboard so the

telemetry is actually looked at, not just stored.

Delivery follows a disciplined **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model: discover what exists and what emits telemetry, design the workspace and author the artifacts, apply everything to a **pilot resource group** first, validate that logs land and alerts fire, then expand the diagnostics policy and alerts across the estate.

What's included

Full documentation (overview, requirements, ordered implementation, testing matrix, troubleshooting, best practices) plus **five deployable artifacts**: a read-only **discovery** script, the core **Bicep** foundation (`OBSAZMON-Foundation.bicep` — workspace, action group, alert rules, diagnostics policy), a **validation** script, a curated **KQL query pack** (`OBSAZMON-Queries.kql`), and a **coverage/ingestion report** script. Everything read-only or **what-if / pilot-first**; no secrets in code.

Business value

- **Fewer surprises** — problems page you with context instead of arriving as an outage.
- **Coverage that scales** — diagnostics apply by **policy**, so new resources are monitored by default.
- **Cost-aware** — deliberate retention, table tiers, and "collect only what you'll act on."
- **Auditable & visual** — one workspace, curated queries, and dashboards ops and execs can read.

Effort and boundary

A competent admin comfortable with the Azure portal and PowerShell can stand up the pilot foundation (workspace, action group, one metric alert, one log alert, diagnostics on a pilot resource group) in a few focused sessions, then expand estate-wide. This guide provides the **technical foundation and the automation**. Your organization's **alert thresholds, on-call routing, retention duration, and what telemetry is worth collecting** are operational decisions the guide wires up but does not decide for you.

> ***Version & pricing note:*** *Azure Monitor features, agent/DCR behavior, table plans (Analytics / Basic / Auxiliary), and* ***ingestion & retention pricing*** *change over time, and some capabilities are region- or SKU-dependent. Every such point in this solution is flagged* ***"verify"*** *— confirm current availability, tier, and pricing in* ***your*** *tenant before broad rollout. *Verify current availability in your tenant.**

1. Solution Overview — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards

What this solution is

Azure Monitor is Azure's built-in observability platform; **Log Analytics** is the workspace where its log and metric telemetry is stored and queried. This solution builds the **ops/platform observability foundation** on top of them: one designed workspace, diagnostic collection applied at scale by policy, alert rules that route to the right team, guest/container insight, and the workbooks and dashboards that make the data usable.

This is **operational (platform) monitoring** — is the estate healthy, are resources performing, did something break — and is deliberately **distinct from a security SIEM** (Microsoft Sentinel). Sentinel answers "are we under attack"; Azure Monitor answers "is it up and healthy." They can share a workspace, but this solution builds the **ops** layer.

How the technology works

- **Telemetry sources.** Azure resources emit **platform metrics** (numeric, near-real-time) and

resource logs (structured events — the categories differ per resource type). VMs and containers additionally emit **guest OS performance** and **container** telemetry via an agent.

- **Diagnostic settings.** A **diagnostic setting** on a resource is the switch that says "send these

log categories and metrics to *this* Log Analytics workspace." Setting these one-by-one does not scale, so this solution assigns them by **Azure Policy** — the `DeployIfNotExists` policy effect creates the diagnostic setting automatically on every in-scope resource (existing and new).

• **Log Analytics workspace.** A workspace stores the telemetry in **tables** you query with **KQL**

(Kusto Query Language). Each table has a **plan/tier** (Analytics / Basic / Auxiliary — *verify current tiers*) and the workspace has a **retention** setting and **RBAC** — these are the cost/access levers.

• **Alerts. Metric alerts** evaluate a platform-metric condition continuously and cheaply; **log

(scheduled query) alerts **run a KQL query on a schedule and fire when the result crosses a threshold. Both fire into an** action group** — the reusable bundle of who/what to notify (email, SMS, Teams/Slack via webhook, PagerDuty, an Azure Function/Logic App/runbook).

• **Insights. VM insights** and **container insights** are curated collection + workbook experiences

driven by the **Azure Monitor Agent (AMA)** and **Data Collection Rules (DCRs)** — the modern replacement for the retired Log Analytics agent (*verify agent status in your tenant*).

• **Workbooks & dashboards. Workbooks** are interactive, parameterized reports built on KQL and

metrics; **Azure dashboards** are pinned tiles for at-a-glance views.

What this solution builds (the components)

1. **Log Analytics workspace foundation**

• One workspace `log-obsazmon-platform` in `rg-obsazmon-monitor`, with a decided **retention**,

daily cap (optional cost guard), and **RBAC** (Log Analytics Reader / Contributor to the right groups). *Table tiers and retention are cost decisions — flagged verify.*

1. **Diagnostics at scale via Azure Policy**

• A **built-in policy initiative** (or per-type policies) assigned at the subscription/management-group

scope — `OBSAZMON-Policy-Diagnostics-Subscription` — with `DeployIfNotExists` so resource logs and platform metrics flow to the workspace automatically, plus the **remediation** task for existing resources.

1. **Alerts + action groups**

• An action group `OBSAZMON-AG-Platform-Ops` (email + Teams/Slack webhook + optional PagerDuty).

• A **metric alert** `OBSAZMON-Alert-VMCPUHigh-Pilot` and a **log alert**

`OBSAZMON-Alert-HeartbeatMissing-Pilot` — starting scoped to the **pilot**.

1. **VM insights / container insights**

• A **Data Collection Rule** `dcr-obsazmon-vminsights` (and the AKS container-insights add-on where

applicable) associating pilot VMs/clusters with the workspace.

1. **Workbooks & dashboards**

• An ops workbook `OBSAZMON-Workbook-OpsOverview` and an executive dashboard

`OBSAZMON-Dashboard-Exec`, driven by the curated `OBSAZMON-Queries.kql` pack.

The standard deployment model

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand*** *— inventory resources and existing diagnostics and decide retention/scope (Assess); deploy the workspace, action group, DCR, and the first alert rules, and stage the diagnostics policy in* ***audit/DoNotEnforce*** *(Configure); apply diagnostics + alerts to a* ***pilot resource group*** *first (Pilot); prove logs land, alerts fire, and dashboards render (Validate); then* ***remediate the policy estate-wide*** *and expand alerts in waves (Expand). Nothing disruptive is enforced fleet-wide before the pilot passes.*

Scope

- Design and deploy **one Log Analytics workspace** as the platform telemetry destination.
- Apply **diagnostic settings at scale** via Azure Policy to route resource logs + metrics.
- Stand up **action groups** and an initial set of **metric + log alert rules**.
- Enable **VM insights / container insights** on the pilot scope.
- Deliver **workbooks + dashboards** and a curated **KQL** pack.

What is out of scope

- **Security monitoring / SIEM** (Microsoft Sentinel analytics rules, UEBA, threat hunting) — that is the **Sentinel** category; this is the ops/platform layer.
- **Application Performance Monitoring internals** (deep Application Insights instrumentation of your app code) — the workspace can host App Insights (workspace-based), but instrumenting application code is a separate development effort.
- The **decisions** of alert thresholds, on-call rotation, retention duration, and which telemetry is worth its ingestion cost — the guide encodes your choices; the choices are yours (and your service owners').
- **Third-party/multicloud** monitoring — for a hybrid/multicloud single pane, see the **Datadog** solution in this category.

Authentication and connections used

- **Az PowerShell / Azure CLI** (Contributor + monitoring rights at the target scope) for discovery, validation, and deployment.
- **Azure Policy managed identity** — the `DeployIfNotExists` assignment uses a **system-assigned managed identity** granted the roles needed to create diagnostic settings (Monitoring Contributor / Log Analytics Contributor) at the assignment scope. No stored secret.
- **Action group channels** — email/SMS need no secret; Teams/Slack use an incoming **webhook URL**; PagerDuty uses an integration key — all treated as secrets (Key Vault / secure input), never in code.

Limitations (be honest with buyers)

• **Ingestion & retention cost money.** Azure Monitor bills on data **ingested** and **retained**;

turning on "everything everywhere" is expensive. Collect what you will act on; use table tiers and a daily cap deliberately. *Verify current pricing/tiers in your tenant.*

• **Feature/agent drift.** The Azure Monitor Agent, DCRs, table plans, and some insight experiences

evolve and vary by region/SKU; all such points are flagged **"verify."**

• **Policy** `DeployIfNotExists` **acts on real resources.** It creates diagnostic settings automatically;

it is staged **DoNotEnforce/audit first** and remediated deliberately, not fired blind estate-wide.

2. Requirements Guide — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards

Verify **every** requirement before you deploy. Each entry: **Description · Why required · How to verify · How to implement if missing.** The category-wide prerequisites (a target cloud/estate with sufficient rights, a telemetry destination, an at-scale rollout channel, IaC tooling + source control, secrets management, an ingestion-cost plan, notification targets, and a pilot scope) are defined once in `global-requirements-observability.md` — confirm those first. The items below are **specific to building the Azure Monitor foundation**.

2.1 Monitoring-capable rights at the target scope

Description: The deploying identity holds **Contributor** (or **Monitoring Contributor** + **Log Analytics Contributor**) at the target **subscription** (or management group), and can **assign Azure Policy** at that scope. **Why required:** You must create a workspace, create diagnostic settings, deploy alert rules and action groups, and assign a `DeployIfNotExists` policy — all of which need resource-write plus policy-assign rights. **How to verify:** `az account show` returns the target subscription; `OBSAZMON-Discovery.ps1` reports your effective roles and confirms you can list Monitor resources and policy assignments. **How to implement if missing:** Have an Owner/User Access Administrator grant Contributor + Monitoring Contributor at the subscription, and Resource Policy Contributor to assign policy.

2.2 A Log Analytics workspace design decision (retention, region, tier)

Description: Decisions for the workspace: **region** (co-locate with the bulk of resources / data-residency needs), **retention** (interactive + optional long-term/archive), whether to use a **daily cap**, and which **table tiers** (Analytics / Basic / Auxiliary) apply. **Why required:** These are the cost and access levers; they are encoded into the Bicep and cannot be defaulted sensibly without your inputs. **How to verify:** A documented decision exists (region, retention days, cap, tiers) that maps to the Bicep parameters. *Verify current tier availability and pricing in your tenant.* **How to implement if missing:** Decide retention and tiers against your cost/compliance posture (this is the customer's decision), then set them as `OBSAZMON-Foundation.bicep` parameters.

2.3 Azure Monitor Agent (AMA) + Data Collection Rule readiness (for VM/container insights)

Description: The **Azure Monitor Agent** extension deployable to the pilot VMs, and rights to create **Data Collection Rules** and associations; for AKS, rights to enable the **container insights** add-on. **Why required:** VM and container insights collect guest/container telemetry via AMA + DCRs (the retired Log Analytics/MMA agent is not used). *Verify AMA availability for your OS/region.* **How to verify:** `Get-AzVMExtension` can be queried on a pilot VM; you can create a DCR in the target region. `OBSAZMON-Discovery.ps1` lists VMs and any existing agents/extensions. **How to implement if missing:** Ensure the pilot VMs are running and reachable, and that AMA is supported for their OS; grant Monitoring Contributor so DCRs and associations can be created.

2.4 Azure Policy managed-identity permissions for DeployIfNotExists

Description: The diagnostics policy assignment uses a **system-assigned managed identity** that must be granted the roles needed to create diagnostic settings — typically **Monitoring Contributor** and **Log Analytics Contributor** at the assignment scope. **Why required:** `DeployIfNotExists` remediation runs *as* the assignment's managed identity; without those roles, remediation fails to create the diagnostic settings. **How to verify:** After assigning the policy, the assignment's identity shows the expected role assignments at scope; a remediation task succeeds on one resource. **How to implement if missing:** Assign the policy with `--location` (so an identity is created) and let Azure grant the declared roles, or grant them manually to the assignment's identity.

2.5 Notification channel details (action group)

Description: The concrete targets for the action group `OBSAZMON-AG-Platform-Ops`: **email address(es)**, a **Teams/Slack incoming webhook URL**, and/or a **PagerDuty integration key**, plus who should receive what. **Why required:** An alert is only useful if it reaches a responder; the action group needs real targets. **How to verify:** The webhook/integration is created in Teams/Slack/PagerDuty and a test message succeeds; the email distribution list exists. **How to implement if missing:** Create the channel(s) and capture the webhook URL / integration key into **secure input / Key Vault** (never in code), then wire them into the action group.

2.6 A defined pilot resource group

Description: A **pilot resource group** (`rg-obsazmon-pilot` or an existing non-critical RG) whose resources are onboarded to diagnostics + alerts first. **Why required:** Diagnostics coverage, alert thresholds, and insight collection are validated on a pilot before the policy is remediated estate-wide. **How to verify:** The pilot RG exists and contains representative resources (a VM, a PaaS resource, ideally an AKS cluster). `OBSAZMON-Discovery.ps1` enumerates it. **How to implement if missing:** Nominate/create a pilot RG with representative resources before scaling. *Alert thresholds and on-call routing are the service owner's decision; this guide provides the wiring.*

2.7 Az PowerShell + Bicep tooling on the operator workstation

Description: **Az PowerShell** (Az.Accounts, Az.Monitor, Az.OperationalInsights, Az.Resources), the **Azure CLI** with **Bicep**, and Git. **Why required:** The read-only helper scripts and the KQL pack use Az; the foundation deploys from **Bicep** (`az deployment ... --template-file OBSAZMON-Foundation.bicep`). **How to verify:** `Get-Module -ListAvailable Az.Monitor` returns a version; `az bicep version` works; `Connect-AzAccount` succeeds. **How to implement if missing:** `Install-Module Az -Scope CurrentUser`; install the Azure CLI and run `az bicep`

`install`; sign in with `Connect-AzAccount` / `az login`.

Checklist

- Category global requirements confirmed (`global-requirements-observability.md`)
- Contributor + Monitoring/Log Analytics Contributor + policy-assign rights — `OBSAZMON-Discovery.ps1`
- Workspace design decided (region, retention, daily cap, table tiers) — *verify tiers/pricing*
- AMA + DCR readiness for pilot VMs / container insights for AKS — *verify AMA support*
- Policy managed-identity roles for `DeployIfNotExists` planned
- Action-group targets (email / Teams-Slack webhook / PagerDuty key) captured to a secret store
- Pilot resource group nominated with representative resources
- Az PowerShell + Azure CLI + Bicep installed and signed in

Global Requirements — Observability (category-level)

These requirements apply to every solution in the Observability category (Azure Monitor / Log Analytics, Datadog, and related telemetry/monitoring platforms). Published once at the category level so listings stay small. Confirm before deploying any observability solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Observability collects telemetry across the estate — collect what you will act on, control ingestion cost deliberately, and roll agents/diagnostics out in a ring before fleet-wide. This is ops/platform monitoring, distinct from security SIEM (see the Sentinel category).*

G1. Target cloud/estate with sufficient rights

Description: The Azure subscription(s), AWS account(s), and/or hosts to be monitored, with rights to configure diagnostics and deploy agents. Why: Monitoring configuration (diagnostic settings, agents, integrations) needs contributor-level access at the target scope. Verify: `az account show` (Azure) / access to the target hosts succeeds with the needed role. Implement: Obtain Contributor (or scoped monitoring) rights at the target scope.

G2. A telemetry destination (workspace / monitoring account)

Description: A Log Analytics workspace (Azure Monitor) or a Datadog organization/account. Why: Telemetry needs a governed destination with retention and access control. Verify: The workspace/Datadog org exists and is access-controlled. Implement: Provision the workspace / Datadog account before wiring sources to it.

G3. Agent / collection rollout channel

Description: A way to deploy agents and collection config at scale — Azure Policy + Azure Monitor Agent / Data Collection Rules, or a config-management/IaC channel for the Datadog agent. Why: Manual per-host

agent installs don't scale and drift. Verify: The rollout channel (Policy/DCR, Intune/CM, Ansible, Terraform) is available. Implement: Establish the at-scale rollout mechanism before onboarding hosts.

G4. IaC tooling and source control

Description: Terraform / Bicep, the relevant CLI (Az / Datadog), and Git. Why: The observability stack is delivered as versioned code so it is repeatable and reviewable. Verify: `terraform version` / `az version` and `git --version` return supported builds. *Verify current provider versions.* Implement: Install and pin the toolchain; store config in Git.

G5. Secrets management

Description: A secure store for API/app keys (Datadog API+APP keys, service principals) — Key Vault / pipeline secrets, never in code. Why: Monitoring keys grant broad read (and some write); committed keys leak. Verify: No keys in the repo; the pipeline reads them from a secret store. Implement: Move keys to Key Vault / secret store; enable secret scanning.

G6. Ingestion cost plan

Description: An understanding of expected data volume and a cost/retention plan (table tiers, sampling, filters). Why: Observability platforms bill on ingestion/retention; uncontrolled collection is expensive. Verify: A volume estimate and retention decision exist. *Verify current pricing/tier availability.* Implement: Define what to collect vs drop, retention, and (Azure) table tiers before broad rollout.

G7. Alerting and notification targets

Description: Notification channels (email, Teams/Slack, PagerDuty) and an on-call/routing model. Why: Alerts are only useful if they reach the right responder with context. Verify: Action groups / Datadog notification integrations and routing are defined. Implement: Configure notification channels and routing before enabling alert rules.

G8. A pilot scope

Description: A defined pilot set of resources/hosts to onboard first. Why: Diagnostics/agents and alert thresholds are validated on a pilot before fleet-wide rollout. Verify: A pilot resource group / host set exists. Implement: Define the pilot scope before scaling collection and alerting. *Alert thresholds and SLOs are the service owner's decision; this guide provides the technical wiring.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards

Follow these steps **in order**. Work in the **Azure portal** (verification + workbooks/dashboards), the **Azure CLI / Az PowerShell** (deployment + scripts), and your **Git repo** (the Bicep + KQL). Names in `code` are the exact objects to create — keep them consistent so the Bicep, policy, alerts, scripts, and dashboards all

refer to the same objects. Feature/tier/pricing points are flagged **"verify."**

Object naming convention: `<Purpose> – <Scope> – <Stage>` (mapped to Azure resource naming). The objects you build:

Object	Name
Monitoring resource group	`rg-obsazmon-monitor`
Pilot resource group	`rg-obsazmon-pilot`
Log Analytics workspace	`log-obsazmon-platform`
Action group	`OBSAZMON-AG-Platform-Ops`
Diagnostics policy assignment	`OBSAZMON-Policy-Diagnostics-Subscription`
Data Collection Rule (VM insights)	`dcr-obsazmon-vminsights`
Metric alert (VM CPU)	`OBSAZMON-Alert-VMCPUHigh-Pilot`
Log alert (heartbeat missing)	`OBSAZMON-Alert-HeartbeatMissing-Pilot`
Ops workbook	`OBSAZMON-Workbook-OpsOverview`
Executive dashboard	`OBSAZMON-Dashboard-Exec`

> ***Model:*** *Assess → Configure →* ***Pilot against*** `rg-obsazmon-pilot` ***first*** *→ Validate → Expand (remediate policy + roll out alerts estate-wide). The diagnostics policy is staged* ***DoNotEnforce / audit first****; nothing disruptive is remediated fleet-wide before the pilot passes. Rollback guidance is at the end.*

Step 1 — Assess: baseline the estate and existing telemetry

1. Run `OBSAZMON-Discovery.ps1` — it inventories subscriptions, resources by type, any **existing Log Analytics workspaces**, **current** diagnostic settings **coverage, existing** action groups **and** alert rules**, and your effective rights.
1. Record the **coverage gap** (resources with no diagnostic setting) — this is what the policy will close.
1. Confirm the requirements checklist (§2), especially **rights**, the **workspace design decision** (region/retention/tiers), and the **notification channel** details.

Step 2 — Configure: deploy the workspace foundation (Bicep)

1. In the portal, create the two resource groups if absent: **Home** → **Resource groups** → **Create** → `rg-obsazmon-monitor` and `rg-obsazmon-pilot` in your chosen region.
1. Deploy the foundation with the provided `OBSAZMON-Foundation.bicep`, which creates the workspace, the action group, and the two pilot alert rules in one reviewed deployment:

```
az deployment group create \
```

```
--resource-group rg-obsazmon-monitor \
--template-file OBSAZMON-Foundation.bicep \
--parameters workspaceName=log-obsazmon-platform \
             retentionDays=90 \
             opsEmail='ops@contoso.com' \
             teamsWebhookUri='<from-secure-input>' \
--what-if           # REVIEW the what-if first
```

Re-run without `--what-if` to apply. *Set `retentionDays`, daily cap, and table tiers from your §2.2 decision — verify current tier availability/pricing.*

1. Confirm in **Monitor** → **Logs**: the workspace `log-obsazmon-platform` exists; set **retention** and

(optionally) a **daily cap** under the workspace's **Usage and estimated costs**.

1. Set workspace **access**: assign **Log Analytics Reader** to the ops group and **Log Analytics

Contributor **to the platform-admin group (workspace** Access control (IAM)**).

Step 3 — Configure: stage the diagnostics policy (audit / DoNotEnforce first)

1. In **Policy** → **Definitions**, find the built-in initiative for routing resource logs to Log Analytics

(e.g. the "Deploy Diagnostic Settings … to Log Analytics workspace" category — *verify the current built-in name/ID*), or use the per-resource-type built-ins.

1. **Assign** it at the **subscription** (or management-group) scope as

`OBSAZMON-Policy-Diagnostics-Subscription`:

- **Parameters:** target workspace = `log-obsazmon-platform`.
- **Enforcement mode:** set to **Disabled (DoNotEnforce)** for now so it **reports** coverage without

auto-creating settings yet.

- **Remediation:** allow the assignment to **create a system-assigned managed identity** and grant it

Monitoring Contributor + **Log Analytics Contributor** at scope (§2.4).

1. Give it time to evaluate, then run `OBSAZMON-Report.ps1` to see the **compliance / coverage** —

which resources *would* get a diagnostic setting. Do **not** remediate estate-wide yet.

Step 4 — Pilot: onboard the pilot resource group

1. **Diagnostics on the pilot:** create a **remediation task** for

`OBSAZMON-Policy-Diagnostics-Subscription` **scoped to** `rg-obsazmon-pilot` **only** (Policy → Remediation → New), or temporarily flip enforcement to **Enabled** limited to the pilot RG. This creates the diagnostic settings on pilot resources → telemetry flows to `log-obsazmon-platform`.

1. **VM insights:** create the DCR `dcr-obsazmon-vminsights` (Monitor → **Data Collection Rules** →

Create, or via the VM insights onboarding) pointing at `log-obsazmon-platform`, and **associate** the pilot VMs — this installs/uses the **Azure Monitor Agent**. For AKS, enable the **container insights** add-on on the pilot cluster.

1. **Confirm telemetry lands:** in **Monitor** → **Logs**, run from `OBSAZMON-Queries.kql`:

```
Heartbeat | summarize LastBeat = max(TimeGenerated) by Computer | order by LastBeat desc
```

Pilot VMs should appear with a recent heartbeat; `AzureDiagnostics` / resource-specific tables should show pilot PaaS logs.

Step 5 — Pilot: wire alerts + action group

1. Confirm the action group `OBSAZMON-AG-Platform-Ops` (created by the Bicep) has the right

email, **Teams/Slack webhook**, and optional **PagerDuty** targets; send a **test notification** (action group → **Test**).

1. Confirm the two pilot alert rules (created by the Bicep), **scoped to** `rg-obsazmon-pilot`:

• `OBSAZMON-Alert-VMCPUHigh-Pilot` — a **metric alert** on VM `Percentage CPU` over your

threshold for N minutes → `OBSAZMON-AG-Platform-Ops`.

• `OBSAZMON-Alert-HeartbeatMissing-Pilot` — a **log (scheduled query) alert** on the `Heartbeat`

table detecting a VM that has stopped reporting → `OBSAZMON-AG-Platform-Ops`.
Thresholds/evaluation windows are your operational decision — set them to your SLOs.

1. **Prove an alert fires:** stress a pilot VM's CPU (or lower the threshold briefly) and confirm the

alert fires and the notification arrives with context. Reset the threshold afterward.

Step 6 — Validate the pilot (the gates)

Run `OBSAZMON-Validate.ps1` and `OBSAZMON-Report.ps1` and confirm the four gates in §5 pass: telemetry lands, diagnostics cover the pilot, an alert fires and notifies, and the workbook/dashboard render. **Pass** = all gates green → proceed to Expand. **Fail** = fix before scaling (see §6).

Step 7 — Build the workbooks and dashboards

1. **Ops workbook:** Monitor → **Workbooks** → **New**, build `OBSAZMON-Workbook-OpsOverview` using queries

from `OBSAZMON-Queries.kql` (heartbeat/health, top resources by ingestion, alert summary, VM performance). **Save** it to `rg-obsazmon-monitor`.

1. **Executive dashboard:** create an Azure **shared dashboard** `OBSAZMON-Dashboard-Exec`, pin the

key workbook tiles and a couple of metric charts, and **share** it (RBAC) with leadership.

1. Confirm both render with real pilot data.

Step 8 — Expand: remediate diagnostics + roll out alerts estate-wide

1. **Promote the diagnostics policy:** once the pilot is clean, flip

`OBSAZMON-Policy-Diagnostics-Subscription` enforcement to **Enabled** and run an **estate-wide remediation task** — in **waves** (by resource group / resource type), re-running `OBSAZMON-Report.ps1` after each wave to confirm coverage climbs and ingestion stays within your cost plan.

1. **Expand alerts:** widen the alert-rule scope from `rg-obsazmon-pilot` to broader scopes (or clone the rules per critical service), keeping the same action group and naming pattern. Add resource-health and service-specific alerts as needed.

1. **Enable insights** on the remaining VMs/clusters by associating them to `dcr-obsazmon-vminsights` (or per-environment DCRs).

1. Re-run `OBSAZMON-Report.ps1` to capture the final **coverage + estimated ingestion** snapshot.

> ***Rollback / safety (any step):*** *Everything is reversible. To stop auto-provisioning, set* `OBSAZMON-Policy-Diagnostics-Subscription` *back to* ***DoNotEnforce*** *(existing settings remain, none new are created). To silence noise,* ***disable*** *an alert rule or the action group rather than deleting it. To cut cost fast, lower* ***retention*** *or set a* ***daily cap*** *on the workspace. Removing a diagnostic setting stops collection for that resource without affecting the resource itself. Do the estate-wide remediation* ***in waves under change control*** *— never remediate the whole estate in one shot (ingestion cost + noise spike).*

Order of operations

`Assess (Discovery) → deploy workspace + action group + pilot alerts (Bicep, what-if first) → stage diagnostics policy in DoNotEnforce → pilot: remediate pilot RG + VM/container insights + prove alerts → validate the 4 gates → build workbook + dashboard → expand: enforce policy + remediate estate-wide in waves + widen alerts`

5. Testing & Validation — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards

Validate against the **pilot resource group** (`rg-obsazmon-pilot`) first. Each test has a **pass/fail** signal. Do not remediate the diagnostics policy estate-wide or widen alerts until all four gates pass. Reference scripts/queries are named in each row.

Test matrix

#	Gate	What you do	Pass signal	Fail signal
1	**G1 Telemetry lands**	Run the `Heartbeat` and resource-log queries from `OBSAZMON-Queries.kql` in Monitor → Logs	Pilot VMs show a recent **heartbeat**; pilot PaaS **resource logs** appear in the workspace	Tables empty / no recent data

#	Gate	What you do	Pass signal	Fail signal
2	**G1 Workspace healthy**	`OBSAZMON-Validate.ps1`	Workspace `log-obsazmon-platform` exists, retention set, RBAC applied	Workspace missing / open access / no retention
3	**G2 Diagnostics cover pilot**	`OBSAZMON-Report.ps1` after the pilot remediation	Pilot resources have a **diagnostic setting** → **workspace**; coverage % reported	Pilot resources still uncovered
4	**G2 Policy staged safely**	Check `OBSAZMON-Policy-Diagnostics-Subscription`	Assigned at scope, **DoNotEnforce** (estate) with pilot remediated; managed identity has roles	Policy enforcing estate-wide already / identity lacks roles
5	**G3 Metric alert fires**	Breach the CPU threshold on a pilot VM (or lower it briefly)	`OBSAZMON-Alert-VMCPUHigh-Pilot` fires; notification arrives via `OBSAZMON-AG-Platform-Ops`	Alert never fires / no notification
6	**G3 Log alert fires**	Stop the AMA/heartbeat on a pilot VM (or simulate)	`OBSAZMON-Alert-HeartbeatMissing-Pilot` fires with context	No alert on a silent VM
7	**G3 Action group reaches responder**	Action group → **Test** (email + webhook + PagerDuty)	Test message received on **every** channel	A channel does not receive
8	**G4 Workbook renders**	Open `OBSAZMON-Workbook-OpsOverview`	Tiles populate with real pilot data	Empty / query errors
9	**G4 Dashboard renders**	Open `OBSAZMON-Dashboard-Exec`	Pinned tiles show current data; shared with the right group	Blank / not shared

Automated checks

• `OBSAZMON-Discovery.ps1` — inventories resources, existing workspaces, diagnostic-settings

coverage, action groups, and alert rules; use it to confirm the pre-state and the coverage gap. **Read-only.**

• `OBSAZMON-Validate.ps1` — checks the deployed foundation: workspace exists with retention + RBAC,

the action group has channels, the pilot alert rules exist and are enabled, and the DCR is associated to pilot VMs. **Read-only.**

• `OBSAZMON-Report.ps1` — reports **diagnostics coverage** (% of in-scope resources with a setting →

the workspace), policy **compliance**, and **estimated ingestion** by table/resource for the cost plan. **Read-only.**

• `OBSAZMON-Queries.kql` — the curated KQL used by the tests, workbook, and dashboard (heartbeat,

resource logs, alert summary, top ingesters, VM performance).

Evidence to capture

- A `Heartbeat` query result showing pilot VMs reporting.
- `OBSAZMON-Report.ps1` coverage CSV (before vs after the pilot remediation).
- A screenshot of the fired **CPU** and **heartbeat** alerts and the received notifications.
- The action-group **Test** confirmation on each channel.
- Screenshots of `OBSAZMON-Workbook-OpsOverview` and `OBSAZMON-Dashboard-Exec` populated.

Exit criteria (before Expand / estate-wide remediation)

- G1 — telemetry lands: pilot heartbeats + resource logs in `log-obsazmon-platform`
- G2 — diagnostics cover the pilot; policy staged **DoNotEnforce** estate-wide with roles granted
- G3 — both alert rules fire and notify through the action group on every channel
- G4 — ops workbook and exec dashboard render with real data
- `OBSAZMON-Validate.ps1` and `OBSAZMON-Report.ps1` run clean; ingestion within the cost plan

6. Troubleshooting — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards

Symptom → root cause → fix for the failures you are most likely to hit. Re-run `OBSAZMON-Validate.ps1` and `OBSAZMON-Report.ps1` after each fix to confirm. Work on the **pilot** first.

No telemetry in the workspace / tables empty

- **Symptom:** `Heartbeat` or resource-log queries return nothing for pilot resources.

• **Cause:** No diagnostic setting on the resource (policy not remediated for the pilot), the DCR isn't associated / AMA not installed, or you are querying too soon (ingestion latency).

• **Fix:** Confirm the pilot resources have a **diagnostic setting** → `log-obsazmon-platform` (portal: resource → Diagnostic settings; or `OBSAZMON-Report.ps1`). For VMs, confirm the **Azure Monitor Agent** extension is present and the VM is **associated** to `dcr-obsazmon-vminsights`. Allow a few minutes for first ingestion. Verify you are querying the **right workspace**.

Policy assigned but nothing is being deployed/remediated

• **Symptom:** `OBSAZMON-Policy-Diagnostics-Subscription` shows resources non-compliant but no diagnostic settings get created.

• **Cause:** Enforcement is **DoNotEnforce** (by design at first), **no remediation task** has run, or the assignment's **managed identity lacks roles**.

• **Fix:** This is expected while staged. To act: create a **remediation task** (scoped to the pilot first), ensure enforcement is **Enabled** for that scope, and confirm the assignment's **system-assigned managed identity** holds **Monitoring Contributor** + **Log Analytics Contributor** at scope (§2.4). Re-assign with `--location` if no identity was created.

Metric alert never fires

• **Symptom:** CPU clearly high on a pilot VM but `OBSAZMON-Alert-VMCPUHigh-Pilot` stays quiet.

• **Cause:** Wrong **scope/resource**, threshold/aggregation/window too lax, the rule is **disabled**, or the metric namespace is wrong for the resource type.

• **Fix:** Confirm the rule's **scope** includes the pilot VM, the **signal** is `Percentage CPU` with a sensible **aggregation** (Avg/Max) and **evaluation window**, and the rule is **Enabled**. Lower the threshold briefly to force a fire, then reset.

Log (scheduled query) alert never fires

• **Symptom:** A pilot VM stopped reporting but `OBSAZMON-Alert-HeartbeatMissing-Pilot` didn't fire.

• **Cause:** The KQL query/threshold/time-window doesn't match reality, or the alert's evaluation frequency is longer than your patience.

• **Fix:** Run the alert's KQL manually in **Monitor** → **Logs** and confirm it returns rows for the silent VM; align the **threshold** (e.g. `count == 0` over the last N minutes) and **frequency** to the heartbeat interval. Check the workspace the rule targets.

Notifications don't arrive

• **Symptom:** Alert shows "fired" but no email/Teams/Slack/PagerDuty message.

• **Cause:** Action group has the wrong/expired **webhook URL** or **integration key**, email caught in spam, or the alert isn't linked to `OBSAZMON-AG-Platform-Ops`.

• **Fix:** Use the action group's **Test** to send to each channel and fix the failing one; regenerate a stale webhook/PagerDuty key (store it in Key Vault); confirm the alert rule's **action group** is set.

Ingestion cost spikes unexpectedly

• **Symptom:** Workspace bill or data volume jumps after a rollout wave.

• **Cause:** A verbose log category enabled estate-wide, a chatty resource type, or high-cardinality custom logs on the Analytics tier.

• **Fix:** Run `OBSAZMON-Report.ps1` **top-ingesters**; drop unneeded log categories, move high-volume low-query data to **Basic/Auxiliary** tier, and set a **daily cap**. Remediate future waves more selectively. *Verify current tier/pricing.*

VM insights / container insights not populating

• **Symptom:** The VM insights map or container insights views are empty.

• **Cause:** AMA not deployed, VM/cluster not associated to a DCR, the container-insights add-on not enabled, or an outbound-network/private-link restriction blocking the agent.

• **Fix:** Confirm the **AMA extension** on the VM and its **DCR association**; for AKS confirm the **container insights** add-on; verify the agent can reach the required Azure Monitor endpoints (proxy/private-link config). *Verify AMA availability for the OS/region.*

Workbook or dashboard shows errors / no data

• **Symptom:** Tiles error or stay blank.

• **Cause:** Query references a table with no data yet, a workspace/resource parameter is unset, or the viewer lacks **Log Analytics Reader**.

• **Fix:** Confirm the underlying tables have pilot data (Gate 1), set the workbook's **workspace/scope parameters**, **and grant viewers** Log Analytics Reader**. Validate each query from `OBSAZMON-Queries.kql` standalone first.

"Access denied" running the scripts or assigning policy

• **Symptom:** `OBSAZMON-Discovery.ps1`/deployment fails, or you can't assign the policy.

• **Cause:** Missing Contributor / Monitoring Contributor / Resource Policy Contributor at the scope, or wrong subscription context.

• **Fix:** `Set-AzContext` to the right subscription; have an Owner grant the roles in §2.1; re-run.

If a failure isn't covered here, capture the resource, the workspace, the exact query/rule, and the `OBSAZMON-Validate.ps1` / `OBSAZMON-Report.ps1` output before changing anything — most issues are a missing diagnostic setting, a mis-scoped alert, or a stale notification channel.

7. Best Practices — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards

Keep the foundation clean, cheap, and trustworthy. These are the habits that separate an observability platform people rely on from an expensive log dump nobody reads.

Collect what you will act on

• **Every log category has a cost.** Don't enable "all logs" on every resource reflexively — route the categories you would actually query or alert on. Use `OBSAZMON-Report.ps1` to find the **top ingesters** and question whether that volume earns its keep.

• **Use table tiers deliberately.** Put high-volume, rarely-queried data on **Basic/Auxiliary** tiers and keep interactive Analytics data lean (*verify current tiers/pricing in your tenant*).

• **Set retention to the requirement, not the maximum.** Interactive retention for operational querying; long-term/archive only where compliance demands it.

• **Guard cost with a daily cap** on non-critical workspaces so a runaway resource can't produce a surprise bill — but understand a cap **drops** data once hit.

One workspace, governed

• Prefer **one (or few) well-governed workspaces** over many ad-hoc ones — cross-resource queries and dashboards need the data in one place. Scope **access with RBAC** (Reader vs Contributor) rather than giving everyone full workspace rights.

• Keep the **ops** telemetry here and let the **security** team run Sentinel (which can share the workspace) — don't blur the two roles.

Diagnostics by policy, not by hand

• Apply diagnostic settings with **Azure Policy** `DeployIfNotExists` so **new** resources are covered automatically — hand-clicked settings drift and get forgotten.

• **Stage it:** DoNotEnforce/audit → pilot remediation → estate-wide remediation **in waves**. Never remediate the whole estate in one action (cost + noise spike).

Alerts that page, not spam

• **Tune before you widen.** Start alerts on the pilot; only expand thresholds that proved meaningful.

An alert that fires constantly gets muted, and then the real one is missed.

• **Route with action groups, not inboxes.** One reusable action group per responder team, with Teams/Slack + on-call (PagerDuty) — not a personal mailbox.

• **Prefer metric alerts** for simple numeric conditions (cheaper, faster) and **log alerts** for cross-resource / KQL logic. Give every rule a **severity** and a clear name (`<Purpose>-<Scope>-<Stage>`).

• **Use alert processing rules** for suppression during maintenance windows instead of disabling rules and forgetting to re-enable them.

Secrets and access hygiene

• Webhook URLs and PagerDuty keys are **secrets** — store them in Key Vault / secure input, never in the Bicep or a committed file.

• The diagnostics policy's **managed identity** should hold only the roles it needs (Monitoring / Log Analytics Contributor) at the assignment scope — no broader.

Version & cost discipline

• The **Azure Monitor Agent**, DCRs, table plans, and pricing **change** — re-verify before a big rollout, and keep the Bicep parameterized so a tier/retention change is a one-line edit.

• Review ingestion **monthly** with `OBSAZMON-Report.ps1`; a creeping bill is usually one noisy new resource type.

Where to go next (adjacent catalog areas)

• **Datadog Deployment for Hybrid & Cloud Estates** (this **Observability** category) — when you need a single pane over **Azure + AWS + on-prem**, not just Azure.

• **Microsoft Sentinel** (the **Sentinel / security** category) — layer **security** analytics, UEBA, and threat hunting on top of (or sharing) this workspace.

• **Azure Landing Zone (Bicep/Terraform)** (the **DevOps / IaC** category) — bake this diagnostics policy and a central workspace into the landing zone so every new subscription is observed by default.

• **Azure Policy governance** solutions — extend the same `DeployIfNotExists` pattern to tagging, security baselines, and cost guardrails.

8. Deliverables Summary — Azure Monitor & Log Analytics: Foundation, Alerts & Dashboards

What you receive

A complete, self-guided package to build an **Azure Monitor observability foundation** — a designed Log Analytics workspace, diagnostic settings applied at scale by Azure Policy, metric and log alert rules routed through action groups, VM/container insights, and workbooks + dashboards. Everything is **read-only or what-if / pilot-first**, with **no secrets in code**.

Documentation

• **Executive Summary** — the problem (uncollected telemetry, outage-driven ops), the five-part solution, the outcome, the boundary, and the version/pricing "verify" note.

• **Solution Overview** — how Azure Monitor, diagnostic settings, Log Analytics, alerts/action groups, and insights fit together; the components built; scope and out-of-scope (SIEM/APM/decisions).

• **Requirements Guide** — the Azure-Monitor-specific prerequisites (monitoring rights, workspace design decision, AMA/DCR readiness, policy managed-identity roles, notification channels, a pilot RG, and the Az/Bicep toolchain), each with Verify / Implement, on top of the category globals.

• **Implementation Guide** — ordered steps: assess → deploy the workspace/action group/pilot alerts (Bicep, what-if first) → stage the diagnostics policy (DoNotEnforce) → pilot the RG + insights → prove alerts → validate → build workbook/dashboard → expand estate-wide in waves, with **every object named** using the `<Purpose>-<Scope>-<Stage>` convention.

• **Testing & Validation** — the four gates (telemetry lands, diagnostics cover the pilot, alerts fire and notify, dashboards render) as a pass/fail matrix with the evidence to capture.

• **Troubleshooting** — symptom → root-cause → fix for the common failures (no telemetry, policy not remediating, alerts not firing, notifications not arriving, cost spikes, agent/DCR issues).

• **Best Practices** — collect-what-you-act-on, table tiers/retention, diagnostics-by-policy, alert-tuning, secrets hygiene, and where to go next.

Deployable artifacts — read-only / what-if first, no secrets

Artifact	Purpose	Default mode
`OBSAZMON-Discovery.ps1`	Inventory resources, existing workspaces, diagnostic-settings coverage, action groups, and alert rules; confirm rights and the coverage gap	Read-only
`OBSAZMON-Foundation.bicep`	The core foundation: Log Analytics workspace (retention/RBAC), action group, and the two pilot alert rules — parameterized, deploy **what-if first**	Declarative (what-if before apply)

Artifact	Purpose	Default mode
`OBSAZMON-Validate.ps1`	Post-deploy checks: workspace + retention + RBAC, action-group channels, pilot alert rules enabled, DCR associated	Read-only
`OBSAZMON-Queries.kql`	Curated KQL pack used by the tests, workbook, and dashboard (heartbeat/health, resource logs, alert summary, top ingesters, VM performance)	Query (read-only)
`OBSAZMON-Report.ps1`	Coverage + cost report: % of resources with diagnostics → workspace, policy compliance, and estimated ingestion by table/resource	Read-only

All PowerShell helpers are **read-only**; the Bicep is deployed `--what-if` **first** and piloted before estate-wide remediation; notification secrets are supplied via **secure input / Key Vault** — **no credentials or secrets live in the artifacts**.

Outcomes

- One **governed Log Analytics workspace** as the platform telemetry destination.
- **Diagnostics coverage that scales** — applied by policy so new resources are monitored by default.
- **Actionable alerts** — metric + log rules that page the right team with context, not an inbox of noise.
- **Visibility** — an ops workbook and an executive dashboard driven by a curated KQL pack.
- A **pilot-first**, reversible rollout (DoNotEnforce ↔ Enforce, disable ↔ enable) with a documented rollback and a cost report.

Assumptions & boundaries

- **Feature/agent/tier/pricing** points are flagged **"verify"** — confirm current availability and cost in your tenant before broad rollout.
- **Alert thresholds, on-call routing, retention duration, and what telemetry to collect** are the customer's/service-owner's decisions — the guide wires them up.
- **Security SIEM** (Sentinel) and **deep application instrumentation** (App Insights code work) are out of scope; **multicloud** single-pane is the Datadog solution.

> *Delivered as a **Complete Solution Package (PDF + Word + scripts)** — the full documentation set plus all five deployable artifacts (Bicep, KQL, and PowerShell helpers) in a single ZIP, yours to keep and reuse on every rollout.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OBSAZMON-Discovery.ps1

Purpose: Baselines the observability starting point before you build the foundation: subscriptions and resources by type, any existing Log Analytics workspaces, current diagnostic-settings coverage (which resources already send logs and where), existing action groups and alert rules, and your effective rights. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OBSAZMON-Discovery.ps1 -CsvFolder C:\Temp\obsazmon Requires Az PowerShell (Az.Accounts, Az.Resources, Az.Monitor, Az.OperationalInsights) and Connect-AzAccount. Reports on the current subscription context (Set-AzContext to change it). EXPECTED RESULTS Console tables + optional CSVs: resource counts by type, workspaces, diagnostic-settings coverage (the gap the policy will close), action groups, and alert rules. Use it to plan retention, the pilot scope, and the diagnostics policy. .NOTES White-label. Read-only against Azure Resource Manager / Azure Monitor. No credentials stored. .EXAMPLE .\OBSAZMON-Discovery.ps1 -CsvFolder C:\Temp\obsazmon > [CmdletBinding()] param([string]$CsvFolder)

How to run: powershell -ExecutionPolicy Bypass -File .\OBSAZMON-Discovery.ps1 -CsvFolder C:\Temp\obsazmon Requires Az PowerShell (Az.Accounts, Az.Resources, Az.Monitor, Az.OperationalInsights) and Connect-AzAccount. Reports on the current subscription context (Set-AzContext to change it). EXPECTED RESULTS Console tables + optional CSVs: resource counts by type, workspaces, diagnostic-settings coverage (the gap the policy will close), action groups, and alert rules. Use it to plan retention, the pilot scope, and the diagnostics policy. .NOTES White-label. Read-only against Azure Resource Manager / Azure Monitor. No credentials stored. .EXAMPLE .\OBSAZMON-Discovery.ps1 -CsvFolder C:\Temp\obsazmon > [CmdletBinding()] param([string]$CsvFolder)

OBSAZMON-Foundation.bicep

Purpose: // Deploys the ops/platform observability foundation in one reviewed deployment: // - Log Analytics workspace (retention + SKU + optional daily cap) -> log-obsazmon-platform // - Action group (email + Teams/Slack webhook + optional PagerDuty) -> OBSAZMON-AG-Platform-Ops // - Metric alert (VM Percentage CPU high, scoped to the PILOT RG) -> OBSAZMON-Alert-VMCPUHigh-Pilot // - Log alert (VM heartbeat missing, scheduled query) -> OBSAZMON-Alert-HeartbeatMissing-Pilot // Diagnostic-settings-at-scale is applied SEPARATELY by Azure Policy (03-implementation Step 3), // staged DoNotEnforce -> pilot -> estate-wide, so it is not baked in here. // // HOW TO RUN (what-if FIRST — never apply blind) // az deployment group create -g rg-obsazmon-monitor --template-file OBSAZMON-Foundation.bicep \ // --parameters workspaceName=log-obsazmon-platform retentionDays=90 opsEmail='ops@contoso.com' \ // teamsWebhookUri='<from-secure-input>' pilotResourceGroupId='<pilot-rg-resourceId>' \ // --what-if // Re-run without --what-if to apply. // // EXPECTED RESULTS // what-if creates the workspace, action group, and the two PILOT-scoped alert rules — and nothing // unexpected. Alert thresholds/windows are parameters — set them to YOUR SLOs. // // NOTES // White-label. No secrets in code: pass webhook/PagerDuty values via secure parameters / Key Vault // reference, never hard-coded. Feature/SKU/pricing points are flagged VERIFY — confirm current // table tiers, retention pricing, and daily-cap behavior in YOUR tenant before broad use. //

OBSAZMON-Queries.kql

Purpose: // The Kusto queries used by the tests (05-testing), the ops workbook // (OBSAZMON-Workbook-OpsOverview), and the executive dashboard // (OBSAZMON-Dashboard-Exec). Run each in Monitor -> Logs against // log-obsazmon-platform, or paste into a workbook query tile. // // HOW TO RUN // Monitor -> Logs -> scope = log-obsazmon-platform -> paste a section -> Run. // Adjust time ranges / thresholds to your SLOs before pinning. // // EXPECTED RESULTS // Read-only queries. No data is modified. Empty results usually mean telemetry // is not yet flowing (see 06-troubleshooting "No telemetry"). // // NOTES // White-label. Table/column names reflect current Azure Monitor schemas — // VERIFY names against your workspace (schemas evolve). // ==========

How to run: // Monitor -> Logs -> scope = log-obsazmon-platform -> paste a section -> Run. // Adjust time ranges / thresholds to your SLOs before pinning. // // EXPECTED RESULTS // Read-only queries. No data is modified. Empty results usually mean telemetry // is not yet flowing (see 06-troubleshooting "No telemetry"). // // NOTES // White-label. Table/column names reflect current Azure Monitor schemas — // VERIFY names against your workspace (schemas evolve). // ==========

OBSAZMON-Report.ps1

Purpose: Produces the governance/cost evidence for the foundation: the percentage of in-scope resources that have a diagnostic setting routing to the workspace (coverage — before/after remediation), the diagnostics policy's compliance state, and the estimated billable ingestion by data type (the top ingesters that drive cost). Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OBSAZMON-Report.ps1 ` -WorkspaceName log-obsazmon-platform -WorkspaceRG rg-obsazmon-monitor -CsvFolder C:\Temp\obsazmon Requires Az PowerShell (incl. Az.OperationalInsights) + Connect-AzAccount. EXPECTED RESULTS Console summary + optional CSVs: coverage %, policy compliance, and billable GB by DataType (last 24h). Compare coverage before vs after each remediation wave; watch the top ingesters against your cost plan. .NOTES White-label. Read-only. No credentials stored. Ingestion figures are estimates — VERIFY against Cost Management / current pricing. .EXAMPLE .\OBSAZMON-Report.ps1 -WorkspaceName log-obsazmon-platform -WorkspaceRG rg-obsazmon-monitor > [CmdletBinding()] param([Parameter(Mandatory)] [string]$WorkspaceName, [Parameter(Mandatory)] [string]$WorkspaceRG, [string]$CsvFolder)

How to run: powershell -ExecutionPolicy Bypass -File .\OBSAZMON-Report.ps1 ` -WorkspaceName log-obsazmon-platform -WorkspaceRG rg-obsazmon-monitor -CsvFolder C:\Temp\obsazmon Requires Az PowerShell (incl. Az.OperationalInsights) + Connect-AzAccount. EXPECTED RESULTS Console summary + optional CSVs: coverage %, policy compliance, and billable GB by DataType (last 24h). Compare coverage before vs after each remediation wave; watch the top ingesters against your cost plan. .NOTES White-label. Read-only. No credentials stored. Ingestion figures are estimates — VERIFY against Cost Management / current pricing. .EXAMPLE .\OBSAZMON-Report.ps1 -WorkspaceName log-obsazmon-platform -WorkspaceRG rg-obsazmon-monitor > [CmdletBinding()] param([Parameter(Mandatory)] [string]$WorkspaceName, [Parameter(Mandatory)] [string]$WorkspaceRG, [string]$CsvFolder)

OBSAZMON-Validate.ps1

Purpose: Post-deployment gate check (05-testing). Confirms: the Log Analytics workspace exists with the expected retention and resource-permission RBAC; the action group has channels; the two pilot alert rules

exist and are enabled; and (optionally) that pilot VMs are associated to the VM insights DCR. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OBSAZMON-Validate.ps1 ` -MonitorRG rg-obsazmon-monitor -WorkspaceName log-obsazmon-platform -PilotRG rg-obsazmon-pilot Requires Az PowerShell + Connect-AzAccount. EXPECTED RESULTS PASS/FAIL per gate. All PASS => foundation is ready; proceed to build workbook/dashboard and to expand. Any FAIL => see 06-troubleshooting. .NOTES White-label. Read-only. No credentials stored. .EXAMPLE .\OBSAZMON-Validate.ps1 -MonitorRG rg-obsazmon-monitor -WorkspaceName log-obsazmon-platform -PilotRG rg-obsazmon-pilot > [CmdletBinding()] param([Parameter(Mandatory)] [string]$MonitorRG, [Parameter(Mandatory)] [string]$WorkspaceName, [Parameter(Mandatory)] [string]$PilotRG)

How to run: powershell -ExecutionPolicy Bypass -File .\OBSAZMON-Validate.ps1 ` -MonitorRG rg-obsazmon-monitor -WorkspaceName log-obsazmon-platform -PilotRG rg-obsazmon-pilot Requires Az PowerShell + Connect-AzAccount. EXPECTED RESULTS PASS/FAIL per gate. All PASS => foundation is ready; proceed to build workbook/dashboard and to expand. Any FAIL => see 06-troubleshooting. .NOTES White-label. Read-only. No credentials stored. .EXAMPLE .\OBSAZMON-Validate.ps1 -MonitorRG rg-obsazmon-monitor -WorkspaceName log-obsazmon-platform -PilotRG rg-obsazmon-pilot > [CmdletBinding()] param([Parameter(Mandatory)] [string]$MonitorRG, [Parameter(Mandatory)] [string]$WorkspaceName, [Parameter(Mandatory)] [string]$PilotRG)